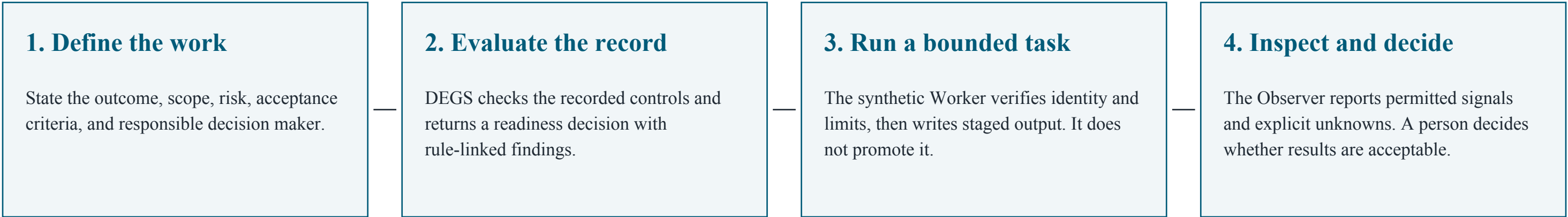




Figure 1
The current staged components and their separate responsibilities



What is present now

The drafts contain the standalone evidence gate, source and policy documents, a synthetic Worker and Observer, and runnable examples. They preserve the core behavior while making its decisions easier to inspect.

What requires a separate decision

A recorded PASS does not approve work. Staged output does not become accepted output by being written. Tests do not qualify hardware or establish operating effectiveness.

Reading the figure. The numbered sequence explains responsibility. It is not an automatically connected execution pipeline. Each component has an explicit interface and limits.

Evidence has a boundary

The strength of the explanation should match the strength of the observation.



The gate evaluates records

A structured task record enters DEGS. Source code does not run inside the gate. The evaluator checks schema, declared risk and authority, required controls, warnings, and stop states. Its output names unmet fields and stable rule IDs.

Figure 2
From an engineering claim to an inspectable readiness result



Human judgment remains necessary

The gate can inspect a recorded status or evidence path without proving the underlying evidence true, independent, complete, or technically sound. Reviewers still judge consequential evidence (Hirshorn et al., 2017; Souppaya et al., 2022).

What the current evidence supports

Observed scenarios are useful evidence. They are not a promise of defect freedom.



DEGS: a useful failure is still a result

The corrected public schema-helper subset matched 241 of 241 examples; 67 core/port tests and five benchmark tests pass. The first run matched 227; its 14 false acceptances traced to one nested boolean/number comparison defect. That observation is preserved. No bypass of the current fixed task schema was demonstrated.

DobeWorks: controlled handling of unusable input

The actual Observer command-line interface (CLI) produced controlled, unusable outcomes for 318 out-of-contract public JSON inputs. Its native fresh and stale examples behaved as expected in the measured run. The repaired harness retains timeout diagnostics, requires stale freshness, and records actual invocation arguments. Six benchmark tests pass.

Current qualification limits

The recorded Worker/Observer suite passed 70 tests. Historical DobeWorks Engineering Assurance Standard (DEAS), Phase 3, and Phase 4 package checks also have inherited identity/scope failures in the draft baseline. The drafts remain private, unqualified development copies. Apple silicon is the tested host scope; other platforms are not claimed.

Credit where it belongs

The foundations span systems engineering, secure development, modular design, verification, and human responsibility. The DEGS draft also preserves a small Sashiko-derived Python review-data component. Muchun Song contributed dismissed-concern and conflict-resolution work; Chris Mason contributed the separate review-prompts foundation. These contributions do not imply a running Sashiko auditing service.

Source basis: the four staged repositories and their recorded artifacts, inspected 11 September 2026. Private device state, activation records, and personal workstreams are outside this edition.

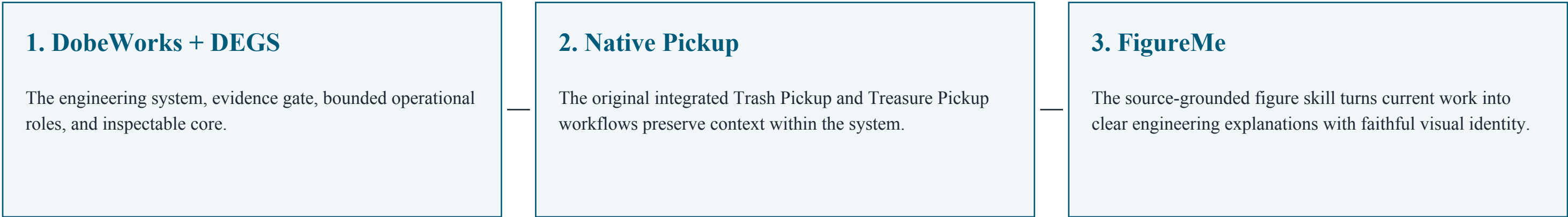
The intended complete release

DobeWorks/DEGS should carry its own useful capabilities with it.



Figure 3

Intended full-bundle composition - packaging remains to be verified



A separate gift for Pickup-only users

The standalone Draft2Staged Trash/Treasure pair serves people who want those context-transfer skills without DobeWorks/DEGS. It is not a substitute dependency for the full system's integrated workflows.

Preserve the useful core

Release preparation should remove personal identifiers while keeping core behavior, source structure, and FigureMe's explanatory character as close to the originals as possible. This diagram records intended composition; it does not claim the current drafts already package every component.

References

- Hirshorn, S. R., Voss, L. D., & Bromley, L. K. (2017). *NASA systems engineering handbook* (NASA/SP-2016-6105 Rev. 2). National Aeronautics and Space Administration. <https://ntrs.nasa.gov/citations/20170001761>
- Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12), 1053-1058. <https://doi.org/10.1145/361598.361623>
- Souppaya, M., Scarfone, K., & Dodson, D. (2022). *Secure software development framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities* (NIST Special Publication 800-218). <https://doi.org/10.6028/NIST.SP.800-218>